

String-Related Interview Questions with Explanations and Examples

1. What is the Difference Between String, StringBuffer, and StringBuilder?

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-Safety	Not thread-safe	Thread-safe (synchronized)	Not thread-safe
Performance	Slow for modifications	Slower due to synchronization	Faster for single-threaded scenarios
Use Case	Use when content is constant	Use in multi-threaded applications	Use for single-threaded mutable strings

Example Code:

```
public class StringTypesExample {
    public static void main(String[] args) {
        // String
        String str = "Hello";
        str.concat(" World"); // Doesn't modify original string
        System.out.println(str); // Outputs: Hello

        // StringBuffer
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World"); // Modifies original object
        System.out.println(sb); // Outputs: Hello World

        // StringBuilder
        StringBuilder sbuilder = new StringBuilder("Hello");
        sbuilder.append(" World"); // Modifies original object
        System.out.println(sbuilder); // Outputs: Hello World
    }
}
```

2. Why Are String Objects Immutable?

- **Security:** Immutable strings prevent data tampering (e.g., when used in ClassLoaders, network connections, etc.).
- **Caching:** Strings are cached in the string pool. Immutability ensures their consistent use across the application.
- **Thread-Safety:** Immutable strings are inherently thread-safe, as their state cannot change once created.

Example:

```
String str1 = "Hello";
```



+91-9130502135 +91-8484831616

OUR TRENDING COURSES

- Java Fullstack Development
- Software Testing (Java-Selenium)
- UI/UX Development
- Cloud Computing

TESTING SHASTRA

WE ARE PUNE'S BEST IT TRAINING INSTITUTE

```
String str2 = str1;
str1 = "World"; // str2 still points to "Hello"
System.out.println(str2); // Outputs: Hello
```

3. What Will You Use to Store Passwords: `char[]` or `String`? Why?

Answer: Use a `char[]` instead of `String` to store passwords.

Reason:

1. Strings are immutable, so once created, their value cannot be changed. Passwords stored in `String` remain in memory until garbage collected, which can lead to security risks if someone dumps memory.
2. `char[]` values can be explicitly cleared after use by overwriting the array.

Example:

```
import java.util.Arrays;

public class PasswordExample {
    public static void main(String[] args) {
        char[] password = {'p', 'a', 's', 's', 'w', 'o', 'r', 'd'};

        // Clear password after use
        Arrays.fill(password, '*');
        System.out.println("Password cleared: " +
Arrays.toString(password));
    }
}
```

4. What Happens When You Create a `String` Object Using Literals and the `new` Keyword?

1. **Using String Literals:**
 - When a string is created using literals, it is stored in the **string pool**.
 - If another string with the same value exists, the same reference is reused.
2. **Using `new` Keyword:**
 - Creates a new object in the **heap memory**.
 - Does not check the string pool for existing strings with the same value.

Example:

```
public class StringCreationExample {
    public static void main(String[] args) {
        // String literal
        String str1 = "Hello";
        String str2 = "Hello"; // Points to the same reference as str1
        System.out.println(str1 == str2); // true

        // Using new keyword
        String str3 = new String("Hello");
        System.out.println(str1 == str3); // false
    }
}
```

5. Where Did You Use `String` and `StringBuffer` in Your Project?

`String` Usage:

1. **Data Storage:** Used to store constant data like file paths, user IDs, or configuration properties.
2. **Validation:** For user input validations like email format or password strength.
3. **APIs:** Used as parameters in API calls for sending or receiving data.

`StringBuffer` Usage:

1. **Thread-Safe Modifications:** Used in multi-threaded scenarios where a string is frequently modified, like logging in a synchronized environment.
2. **File Handling:** Concatenating file contents while reading in a multi-threaded application.

Example:

```
public class ProjectExample {
    public static void main(String[] args) {
        // Using String for file path
        String filePath = "/user/data/file.txt";

        // Using StringBuffer for multi-threaded logging
        StringBuffer log = new StringBuffer();
        synchronized (log) {
            log.append("Thread-1: Start\n");
            log.append("Thread-2: Processing\n");
            System.out.println(log);
        }
    }
}
```

These answers provide a comprehensive understanding of how and why to use different `String` types in Java. Let me know if you need further clarifications or additions!